
Machine Learning II - Structured Models

Exercise sheet **Introduction to OpenGM inference library, CPLEX as solver: solver parameters, scalability.**

Bogdan Savchynskyy - Bogdan.Savchynskyy@tu-dresden.de

Outlook:

1. Preliminaries:

- Ex. 1-2 Check installation of Python and OpenGM.
- Ex. 3: Familiarize yourself with a simple python code to create graphical models in OpenGM format.

2. Objectives:

- Ex. 4: Check scalability for energy minimization of CPLEX as an out-of-the-shelf LP/ILP solver on randomly generated models.
- Ex. 5: Check scalability of specialized exact energy minimization solvers (ILP).
- Ex. 6: Compare results of approximate solvers: a naive solver and an approximate LP solver.
- Ex. 7: (Optional) Compare visual results.

3. Results: fill out the table.

Exercise 1 (Python installation check).

Type in terminal

```
> python
```

Check the output version - should be 2.7.x

```
>>> import numpy
```

```
>>> import opengm
```

should not return any error messages.

Tutorials for Python:

<https://docs.python.org/2.7/tutorial/>

http://www.tutorialspoint.com/python/python_quick_guide.htm

http://www.slideshare.net/amiable_indian/introduction-to-python

<https://wiki.python.org/moin/BeginnersGuide>

Exercise 2 (OpenGM installation check).

Type in terminal

```
> opengm_min_sum -h
```

should return a short help

OpenGM library and benchmark for energy minimization:

Project web-page <http://hci.iwr.uni-heidelberg.de/opengm2/>

Python examples <https://github.com/homul/opengm/tree/master/src/interfaces/python/examples>

C++ examples <https://github.com/homul/opengm/tree/master/src/examples>

C++ tutorials <https://github.com/homul/opengm/tree/master/src/tutorials>

Exercise 3 (Test models creation).

Familiarize yourself with the simple random model constructor for OpenGM: `model_construct.py`

Check it code, run it to get help:

```
> python model_construct.py -h
```

Run it with different parameters, in particular, with Potts potentials.

Check parameters of the created model with openGM commandline tool:

```
opengm_min_sum --modelinfo -m my_model.h5
```

Exercise 4 (CPLEX as solver. Scalability.).

CPLEX is a commercial out-of-the-shelf ILP solver:

<http://www-01.ibm.com/software/commerce/optimization/cplex-optimizer/>

Generate several models starting from small size/number of labels and gradually increase these values.

Use CPLEX with OpenGM library to obtain optimal labeling.

1. Check parameters of the CPLEX in OpenGM:

```
> opengm_min_sum -a LPCPLEX -h
```
2. Use it as ILP solver

```
> opengm_min_sum -a LPCPLEX -m my_model.h5 -ic -v -p 1...
```
3. and LP solver

```
> opengm_min_sum -a LPCPLEX -m my_model.h5 -v -p 1...
```
4. Use parameters `--timeout` and `--maxTime` to limit the maximal runtime if needed.
5. Perform a series of experiments for 2, 4, 10 labels and gradually increasing number of variables.
6. Check resulting `opengm_result.h5` with `hdfview`:

```
> hdfview opengm_result.h5
```

What is the structure of the OpenGM output?

 - What are the typical problem sizes solvable withing 1-2 minutes in ILP mode? In LP mode?
 - What is more influential: number of labels or grid size?
 - Do the answers change significantly for Potts potentials?
 - Which energy value is lower - in the LP or ILP mode? And bound?

Exercise 5 (Specialized solvers).

- Generate a Potts model with 2 labels of the size 400×400 . Is it solvable with CPLEX?

- And with Max-Flow algorithm
`> opengm_min_sum -a GRAPHCUT -m my_model.h5 -v -p 1`
- Download models from Color segmentation N4 dataset
<http://hci.iwr.uni-heidelberg.de/opengm2/benchmarks/models/zip/color-seg-n.zip>
- Check their size and type with `--modelinfo` parameter of `opengm_min_sum`. Compare to the model description
<http://hci.iwr.uni-heidelberg.de/opengm2/benchmarks/models/model-descriptions/color-seg-n4.pdf>
- Are these model solvable with CPLEX?
- Run CombiLP solver to obtain their exact solutions:
`> opengm_min_sum -a CombiLP --maxIt 1000 --treeAgreeMaxStableIter 100 -v -p 1 ...`

Exercise 6 (Naive solver, LP bound).

1. Proceed tests on the Color segmentation N4 dataset.
2. Familiarize yourself with a naive node-wise solver `naive_solver.py`. Run it to get the energy (and labeling) for several instances (see Table-1) of Color segmentation N4 dataset. Use different `output-file-name.h5` for different problem instances and different algorithms. Save the output h5 files. If h5py package is not installed (`> python -c "import h5py"` should not return any error) comment out the lines after `# Comment the following 3 lines if h5py is not installed`
3. Run TRWS (approximate specialized LP solver) to obtain a rounded LP solution and bound and compare the results.
`> opengm_min_sum -a TRWS -v -p 1 --maxIt 1000 --energy TL1 -o output-file-name.h5 ...`
4. Run CombiLP to obtain an exact ILP solution and compare the results.
`> opengm_min_sum -a CombiLP --maxIt 1000 -v -p 1 --treeAgreeMaxStableIter 100 -o output-file-name.h5 ...`
5. Compare energies of the obtained labelings. How tight is the bound obtained by TRWS?
6. Repeat items 2,3 and 5 for problem instance from the MRF Stereo dataset.
For `tsu-gm` and `ven-gm` use
`> opengm_min_sum -a TRWS -v -p 1 --maxIt 1000 --energy TL2 -o output-file-name.h5 ...`
for `ted-gm`
`> opengm_min_sum -a TRWS -v -p 1 --maxIt 1000 --energy TL1 -o output-file-name.h5 ...`

Exercise 7 ((Optional) Visual Comparison).

1. Check whether packages `h5py` and `matplotlib` are installed:
`> python -c "import h5py"`
`> python -c "import matplotlib"`
should not return any error messages.
In case of errors ask sysadmin to run item 2.1) from the `installation-readme.txt`.

2. Use the output `h5`-files from Exercise 6.
3. Run the python script `show_labeling.py` to transform output labeling into a color image. Check its parameters with
 `> python show_labeling.py -h`
 Example of use:
 `> python show_labeling.py -V 288 -H 384 -v -C -f output-file-name.h5`
 Size of the models:
 all from `color-seg-n4`: -V 240 -H 320
 tsu-gm: -V 288 -H 384
 ven-gm: -V 383 -H 434
 ted-gm: -V 375 -H 450
4. Compare visual results of all solvers.